

PilotFish RESTful Web Service Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **REST API front door** on your PilotFish server. You configure a service name and supported resources; HTTP clients call URLs under that service, and each valid request starts an integration transaction. If synchronous mode is on, the route's output becomes the HTTP response body.

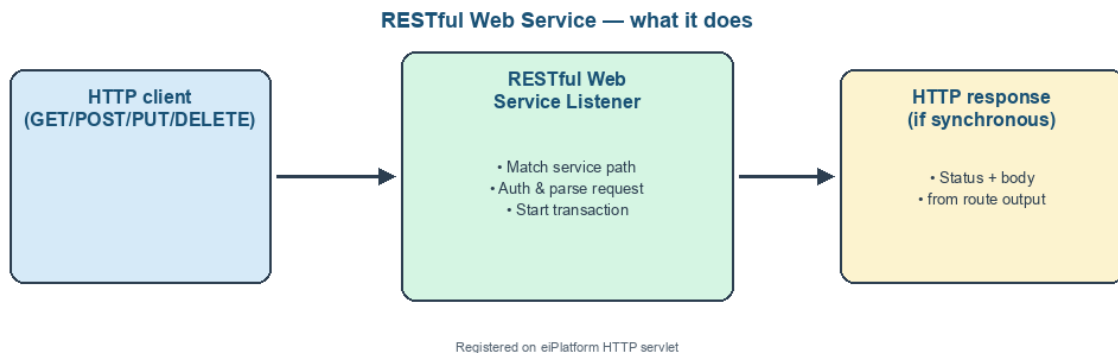


Figure 1: At a glance (plain English)

- **Exposes** REST endpoints via the eiPlatform HTTP servlet.
- **Authenticates** optional Basic auth or legacy token query parameter.
- **Maps** path, method, headers, and parameters to transaction attributes.
- **Returns** HTTP status and body when synchronous response is enabled.

Purpose

Deep dive into the **RESTful Web Service** listener in **eip.war.hs.26R1.11**: servlet registration, path parsing, auth, synchronous HTTP mapping, and configuration.

Provenance	Value
WAR	eip.war.hs.26R1.11
Module JAR	xcs-modules-http-1.0.3.jar
Core JAR	xcs-core-1.2.6.jar
Decompiler	CFR 0.152
Implementation class	com.pilotfish.eip.modules.http.rest.RESTfulWebServiceListener
Decompiled path	war-pipeline/decompiled/eip-26R1.11/.../RESTfulWebServiceListener.class
Listener type string	RESTful Web Service
Description	Monitors a specified local HTTP path for RESTful requests.
Help ID	console.listener.restful_web_service

Related classes:

Class	Role
RESTfulWebServiceListener	Request handling, auth, dispatch
HttpRequestServlet	Servlet routing, listener registry
RESTfulWebServiceTransport	Outbound REST counterpart

Servlet dependency registered: `ServletDependencyRegistry.registerDependency(RESTfulWebServiceListener, HttpRequestServlet)`.

1. High-level behavior

1. **startListening**: validates config; registers listener with `HttpRequestServlet.addRequestListener(this, serviceName)`. Duplicate service name logs error and does not listen.
2. **HTTP request** hits servlet → `requestReceived(RESTHttpRequestEvent)`:
 - Optional Basic auth or legacy token check
 - Require SSL gate
 - Read body stream → inner `requestReceived(InputStream, event)`
3. **Method guard**: unsupported verbs return **501** with fixed message.
4. **Path validation**: service name, resource name required or **400**.
5. **Attribute assembly**: headers, query params, servlet objects, REST path parts.
6. **Async**: `notifyListeners(body, attrs)`; returns bytes "pf:default".
7. **Sync**: `dispatchForSynchronousResponseTransactionData` with **Timeout**; maps response stream + `com.pilotfish.HTTPResponseCode` to HTTP response. Timeout → **408**.

```

HTTP -> HttpServletRequest -> RESTfulWebServiceListener
    auth / SSL checks
    parse path: /{service}/{resource}/{id}/{method}
    build TransactionAttributes
    [Synchronous?] dispatch sync : notifyListeners async
    return byte[] HTTP body

```

2. Configuration reference

2.1 Basic

Label	Tag	Default	Notes
REST Service Name	SERVICE_NAME	""	Required; no / in name; URL segment
Supported Resources	SupportedResources	""	Comma list; required
Timeout	Timeout	600	Sync wait seconds

Path pattern (documented in UI): `http://server/service/{ServiceName}/Car/21` — resource list constrains valid resource names.

2.2 Authentication tab

Label	Tag	Default
User Name	USER_NAME	""
Password	PASSWORD	""

When both non-empty, **HTTP Basic** required on requests.

2.3 Supported HTTP Methods tab

Label	Tag	Default
POST	POST_SUPPORTED	true
GET	GET_SUPPORTED	true
PUT	PUT_SUPPORTED	true
DELETE	DELETE_SUPPORTED	true

Unsupported method → 501 + error notification.

2.4 Advanced tab

Label	Tag	Default	Notes
Require SSL	RequireSSL	false	Rejects non-secure with 405
Synchronous	Synchronous	false	Route must produce HTTP response
HTTP Headers	HTTPHeaders	""	Name/value table for sync response headers
Use Legacy Token Parameter	UseLegacyTokenParameter	false	MD5 token in query string

Synchronous shows **HTTP Headers** table (applied via `setResponseHeaders`).

2.5 WADL

Label	Tag	Purpose
WADL file	WadlFile	Returned when request ends with <code>?wadl</code>

`getWadl()` resolves file from configuration.

3. Authentication algorithms

3.1 HTTP Basic

When User Name and Password configured (trimmed non-empty):

1. Requires **Authorization** header with at least two space-separated fields.
2. Scheme must be **Basic** (others → 400).
3. Base64-decodes credentials; must be exactly `user:password`.
4. Mismatch → **401** + error notification.

3.2 Legacy token parameter

When **Use Legacy Token Parameter** true and query contains `token`:

1. Computes MD5 hex of `yyyyMMdd + USER_NAME + PASSWORD`.
2. Extracts token from query (`token=` + 32 hex chars).
3. Mismatch → error notification; returns null body path.

Use only for backward compatibility.

3.3 Require SSL

If true and `!request.isSecure()` → **405** “SSL Required”, empty body.

4. Transaction attributes

Registered attributes (selection):

Attribute	Source
<code>com.pilotfish.ServiceName</code>	Parsed service name from event
<code>com.pilotfish.ResourceName</code>	Resource segment
<code>com.pilotfish.ResourceID</code>	ID segment if present
<code>com.pilotfish.ResourceMethod</code>	Method segment
<code>com.pilotfish.HttpMethodName</code>	HTTP verb
<code>com.pilotfish.RequestPath</code>	<code>request.getPathInfo()</code> (full path)
<code>com.pilotfish.RequestQueryString</code>	Query string
<code>com.pilotfish.Content-Type</code>	Content-Type header
<code>com.pilotfish.ServerName</code>	<code>request.getServerName()</code>
<code>com.pilotfish.ServerPort</code>	Server port
<code>com.pilotfish.RemoteAddr / RemoteHost</code>	Client
<code>com.pilotfish.RequestURL</code>	Full URL
<code>com.pilotfish.Protocol</code>	Protocol
<code>com.pilotfish.Locale</code>	Client locale
<code>com.pilotfish.servlet.request</code>	<code>HttpServletRequest</code> object
<code>com.pilotfish.servlet.response</code>	<code>HttpServletResponse</code> object
<code>com.pilotfish.{HeaderName}</code>	Each HTTP header
<code>com.pilotfish.http.parameter.{name}</code>	Each query/form parameter
<code>com.pilotfish.Synchronous</code>	"true" / "false" string

Response (sync):

Attribute	Role
<code>com.pilotfish.HTTPResponseCode</code>	Numeric status (validated [0-9]+)
<code>com.pilotfish.eip.modules.http.StoredHeaders</code>	Map of extra response headers

Configured **HTTP Headers** table also applied on synchronous responses.

5. Synchronous response mapping

1. Waits up to **Timeout** seconds for route completion.
2. Reads response data stream as body bytes.
3. Parses optional `com.pilotfish.HTTPResponseCode` (default **200**).
4. Applies `StoredHeaders` map and configured HTTP Headers table.
5. Sets servlet response status and returns body.

Failures reading response → **500** with error message bytes.

Timeout with null response → **408**, `notifyListenersOnError(EIPEException("Transaction Timeout"))`.

6. Async mode

`notifyListeners(dataAsBytes, attrs)` then returns UTF-8 bytes of **pf:default** — servlet layer treats this as default async acknowledgment pattern.

7. Worked example

Service Name **orders**, Supported Resources **order,shipment**, Synchronous **true**, Timeout **120**.

POST **https://host/eip/http/orders/order/123/validate** with Basic auth:

1. Listener validates method, auth, resource **order**.
 2. Attributes include **ResourceID=123**, **ResourceMethod=validate**, body as transaction stream.
 3. Route processes and sets **HTTPResponseCode=204**.
 4. Client receives 204 with route output body (if any).
-

8. Known quirks

1. **Duplicate service name** prevents second listener from starting (error logged).
 2. **Async default body pf:default** — clients must not expect business payload in async mode.
 3. **Legacy token** uses date-based MD5 — breaks across midnight UTC/local depending on server date.
 4. **Attribute overwrite:** **RequestPath** first set to configured service name, then overwritten with **pathInfo**.
 5. **ServerName attribute** overwritten with parsed **service name** (not HTTP server host) after initial host assignment.
 6. **TimeoutHandlingMechanism** constant exists in class but not exposed in 26R1.11 descriptor (dead legacy field).
 7. **501 vs 405:** unsupported method vs SSL required use different statuses.
-

9. Operational recommendations

Goal	Settings
Public API	Synchronous true ; set HTTP Headers (CORS, content-type)

Goal	Settings
Internal fire-and-forget	Synchronous false
Secure production	Require SSL true; Basic auth or fronting SSO gateway
API discovery	Configure WADL file
Long processing	Increase Timeout; consider async + polling pattern instead
Unique endpoints	One listener per SERVICE_NAME

10. Inherited AbstractListener options

Full base configuration available (trigger-only, CLI, restart, FIFO, throttling, inactivity). REST listener overrides `startListening` / `stopListening` rather than engage/disengage polling pattern.

11. Source index

Topic	Path
RESTfulWebServiceListener	com/pilotfish/eip/modules/http/rest/RESTfulWebSer
HttpRequestServlet	com/pilotfish/eip/modules/http/HttpRequestServlet

Deep-dive documentation from WAR decompile — 26R1.11 — August 2026.